

系统程序设计 Lab 1 — Part A 实验报告

UNIX I/O 基础性能对比实验

项目	内容
课程	系统程序设计
实验	Lab 1 Part A — UNIX I/O 基础性能对比
姓名	印伟辰
学号	23302010024
日期	2026.5.23

1 实验目的

通过实测与分析，理解以下几种 UNIX/Linux I/O 方式的性能差异及其背后的系统原理：

- 1. 系统调用 `read()` 在不同 `BUFSIZE` 下的读取效率；
- 2. 标准 I/O 库逐字符函数 `getc()` / `fgetc()` 的读取效率，以及宏实现与函数实现的差异；
- 3. 标准库块读取函数 `fread()` 在不同 `BUFSIZE` 下的读取效率；
- 4. 自行实现的带缓冲读取函数 `my_fread()` 的效率表现；
- 5. `write()` 在有无 `O_SYNC` 标志下的写入效率差异。

核心目标是把"实测数据"与"系统调用开销、内核页缓存、用户态缓冲"等原理对应起来，解释为什么会出现这样的性能曲线。

2 实验环境

本次正式数据在云服务器上采集。下表取自程序启动时 `detect_environment()` 打印的环境信息。

项目	取值
操作系统	Linux（据内核版本推断为 Ubuntu 22.04 LTS）
内核版本	5.15.0-126-generic (#136-Ubuntu SMP Wed Nov 6 10:38:22 UTC 2024)
运行环境	虚拟机（检测到 hypervisor CPU 标志；主机名 <code>VM-32-133-ubuntu</code> ，为云服务器实例）
硬件架构	x86_64
处理器	逻辑核心数 4（CPU 型号未由程序采集）
物理内存	未由程序采集
文件系统	ext2/3/4 (<code>statfs</code> magic <code>0xef53</code> ，测试路径 <code>.</code>)，块设备号 252:2

项目	取值
文件系统块大小	4096 字节
内存页大小	4096 字节
磁盘类型	HDD / 旋转磁盘 (<code>rotational=1</code> , 设备 252:2)
编译器	gcc (具体版本未采集)
构建工具	GNU Make
编译选项	<code>-O2 -Wall -Wextra -std=c99</code>

实验环境说明：本实验运行于云虚拟机（检测到 hypervisor CPU 标志，主机名 `VM-32-133-ubuntu`）。需要特别留意以下两点对结果的影响：

- **虚拟化 I/O 路径：**虚拟机的块设备号为 252:2（典型的 virtio/虚拟块设备主设备号），数据要经过 Guest 文件系统 → 虚拟块设备 → Host 存储后端的多层转换，绝对 I/O 性能与延迟不完全反映底层物理盘。
- **`rotational` 标志的可信度有限：**程序读 `/sys/.../queue/rotational` 得到 1，据此判定为 HDD；但该标志由虚拟磁盘上报，云盘底层很可能是 SSD 或网络块存储，因此"HDD"仅为虚拟设备的自我描述，不能据此推断真实介质。后文 `O_SYNC` 实测出的 单次同步落盘延迟约 2.4 ms（见 5.4 节），介于典型 HDD（约 10 ms）与本地 SSD（亚毫秒）之间，也印证了底层是经过虚拟化/网络的块存储。

缓存条件：本次运行未加 `-d` 选项（测试参数中"测前逐出页缓存：否"），即在 ****热缓存（hot cache）**** 条件下采集。这一点对读测试的解读至关重要——读测试的耗时 几乎完全由 CPU 与系统调用开销决定，而非磁盘传输（详见 5.1 节的佐证）。

测试规模：读测试文件 `testfile` 为 512 MB；写测试（无 `O_SYNC`）总量 256 MB；写测试（`O_SYNC`）受 `OSYNC_MAX_WRITES = 4096` 限制，总量上限 16 MB。程序对每项只测一次，下列数据为单次运行结果。

备注：测试程序的开发与冒烟测试在 WSL2（Ubuntu 24.04，内核 6.6.87.2-microsoft- standard-WSL2）环境下完成；本报告性能数据以上述云服务器实测为准。

3 实验设计与方法

3.1 总体设计与程序流程

测试程序为单文件 C 程序 `partA/main.c`，整体流程分为三个阶段：

① 确定版本环境 → ② 分别调用各测试函数 → ③ 聚合并输出结果

- **① 确定版本环境：**`detect_environment()` 通过 `uname()`、`statfs()`、`/sys/dev/block/*/queue/rotational`、`sysconf()` 等接口收集并打印操作系统、内核版本、文件系统类型、磁盘类型等信息（即第 2 节内容）。
- **② 分别调用测试函数：**对 6 个测试项分别实现独立的测试函数，按 `BUFFSIZE` 循环逐一测量。

- ③ 聚合并输出结果：每完成一次测量即打印一行对齐表格，并把结果汇总写入 CSV 文件 `results.csv`，便于后续绘图。

3.2 计时方法

每个测试函数在被测 I/O 循环的前后各取一次"快照"，二者相减得到三种时间：

指标	含义	采集方式
real（实时/墙钟时间）	测量区间实际流逝的时间	<code>clock_gettime(CLOCK_MONOTONIC)</code>
user（用户态 CPU 时间）	进程在用户态消耗的 CPU 时间	<code>getrusage(RUSAGE_SELF)</code> 的 <code>ru_utime</code>
sys（内核态 CPU 时间）	进程执行系统调用、在内核态消耗的 CPU 时间	<code>getrusage(RUSAGE_SELF)</code> 的 <code>ru_stime</code>

三者关系大致为 $real \approx user + sys + \text{阻塞等待时间}$ 。其中 `sys` 直接反映系统调用开销，是分析 `read()/write()` 性能的关键指标；选用 `CLOCK_MONOTONIC` 是为了避免系统 时钟被调整时影响测量。

3.3 BUFSIZE 的选取

按实验要求，对需要扫描缓冲区大小的测试项使用以下 18 种 BUFSIZE（单位：字节）：

```
1, 2, 4, 8, 16, 32, 64, 128, 256, 512,
1024, 2048, 4096, 8192, 16384, 32768, 65536, 16777216
```

取值大体按 2 的幂递增，覆盖从"逐字节"到"远大于文件系统块（4096 字节）"再到 "16 MB 大块"的范围，便于观察性能随缓冲区增大而变化的完整趋势。

3.4 各测试项的实现方法

3.4.1 read()（对应 2.2.1）

以 `open(path, O_RDONLY)` 打开测试文件，在计时区间内循环调用 `read(fd, buf, bufsize)` 直到返回 0（EOF），累计读取字节数。对 18 种 BUFSIZE 各完整读取一遍文件。

3.4.2 getc() / fgetc()（对应 2.2.2）

以 `fopen()` 打开文件，循环调用 `getc(fp)`（宏实现）或 `fgetc(fp)`（函数实现）逐字符读取直到 EOF。两者分别测试，用于对比"宏 vs 函数"的调用开销差异。此项不涉及 BUFSIZE（stdio 的内部缓冲区由库自行管理）。

3.4.3 fread()（对应 2.2.3）

以 `fopen()` 打开文件，循环调用 `fread(buf, 1, bufsize, fp)` 直到读完。此处的 BUFSIZE 是每次 `fread` 请求的字节数；FILE 对象内部仍有一层由 stdio 库维护的缓冲区。对 18 种 BUFSIZE 各测一遍。

3.4.4 my_fread() 的设计与实现（对应 2.2.4）

`my_fread()` 仿照标准库 `fread()` 的工作原理自行实现，内部基于 `read()` 系统调用 维护一个用户态缓冲区。其“文件对象”定义为：

```
typedef struct {
    int      fd;          /* 底层文件描述符      */
    unsigned char *buf;    /* 用户态缓冲区        */
    size_t   bufsize;     /* 缓冲区容量（固定）  */
    size_t   pos;         /* 下一个待读字节的下标 */
    size_t   len;         /* 缓冲区中有效字节数   */
    int      eof;         /* 是否已到文件末尾     */
} MYFILE;
```

`my_fread()` 的核心逻辑：

1. 优先从内部缓冲区 `buf` 中拷贝数据给调用者；
2. 仅当缓冲区耗尽 (`pos == len`) 时，才调用一次 `read(fd, buf, bufsize)` 向缓冲区补充数据；
3. 重复上述过程直到满足请求字节数或到达文件末尾，返回成功读取的完整成员数。

这样，多次小请求只会触发少量 `read()` 系统调用，从而把昂贵的系统调用开销摊薄——这正是标准库 `fread()` 高效的原因。

测试方式：内部缓冲区容量固定为 `BUFSIZ` (8192 字节)，与 `stdio` 默认缓冲区一致；扫描的 18 种 `BUFSIZE` 作为每次 `my_fread` 请求的字节数。这样与 `fread()` 测试 形成对等对比（二者内部缓冲均固定，仅请求大小变化）。

3.4.5 write()（无 O_SYNC，对应 2.2.5）

以 `open(path, O_WRONLY|O_CREAT|O_TRUNC)` 打开文件，循环调用 `write(fd, buf, bufsize)` 直到写满指定总量（默认 256 MB）。此时数据写入内核 页缓存即返回，不强制落盘。对 18 种 `BUFSIZE` 各测一遍。

3.4.6 write()（有 O_SYNC，对应 2.2.6）

打开文件时额外加上 `O_SYNC` 标志，其余与 3.4.5 相同。`O_SYNC` 语义为：每次 `write()` 调用都必须等数据真正落到存储设备后才返回。

实现上的特殊处理：在 `O_SYNC` 且 `BUFSIZE` 很小时，写满几百 MB 需要上千万次 同步落盘，耗时不可接受。因此 `O_SYNC` 测试为每个 `BUFSIZE` 限制最多 `OSYNC_MAX_WRITES = 4096` 次 `write()` 调用，写入总量上限 16 MB；实际写入字节数 随 `BUFSIZE` 变化，记录在结果表的吞吐量与 CSV 的 `bytes` 列中。这样既能观察到 `O_SYNC` 随 `BUFSIZE` 变化的趋势，又把测试时间控制在合理范围内。

3.5 实验方法学（减小误差的措施）

- **测试文件内容：**用伪随机数据填充，避免文件系统/虚拟磁盘对全零数据做稀疏或 压缩优化而影响结果。
- **清除缓存：**读测试受内核页缓存影响极大。建议两种做法之一：
 - 测试前手动执行 `sync && echo 3 | sudo tee /proc/sys/vm/drop_caches`；

- 运行程序时加 `-d` 选项，让程序在每次读测试前用 `posix_fadvise(POSIX_FADV_DONTNEED)` 逐出该文件的页缓存。报告中需注明本次数据是"冷缓存"还是"热缓存"条件下采集的。
- 多次取平均：程序对每个测试项只测一次；建议运行程序 3 次以上，对结果取平均 后再填入下表，以减小偶发抖动的影响。

3.6 编译与运行

```
cd partA
make                                # 生成可执行文件 ioperf

# 正式运行（默认 512MB 读文件，约需数分钟）
sync && echo 3 | sudo tee /proc/sys/vm/drop_caches
./ioperf                            # 结果输出到终端，并汇总写入 results.csv

# 常用选项
./ioperf -h                         # 查看全部选项
./ioperf -s 256 -d                  # 读文件 256MB，每次读测试前逐出页缓存
make clean                          # 清理可执行文件与测试产物
```

运行结束后，`results.csv` 各列为 `test`, `buffsize`, `real`, `user`, `sys`, `bytes`, `throughput_MBps`，可直接导入 Excel / Python(matplotlib) 等工具绘制性能曲线。

绘图：本报告 4.7 节的图由 `partA/plot.py` 读取 `results.csv` 生成，输出到 `partA/fig/`：

```
python3 -m venv .venv && .venv/bin/pip install matplotlib
.venv/bin/python plot.py          # 生成 fig/fig1~fig4.png
```

本报告第 4 节数据为热缓存条件下采集（运行时未加 `-d`，亦未手动 `drop_caches`），即 512 MB 测试文件已驻留内核页缓存。因此读测试反映的是"页缓存→用户缓冲区"的拷贝 与系统调用开销，而非物理磁盘读取速度，这恰好凸显了系统调用与缓冲机制本身的代价。

4 实验结果

数据取自 `results.csv`（热缓存条件，单次运行）。`real/user/sys` 单位为秒，吞吐单位为 MB/s（以 1 MB = 1,000,000 字节计，与程序输出一致）。

4.1 read() 系统调用（不同 BUFFSIZE）

BUFFSIZE（字节）	real(s)	user(s)	sys(s)	吞吐(MB/s)
1	418.512	178.231	240.240	1.2
2	209.718	89.631	120.073	2.4
4	104.735	44.532	60.195	4.9
8	52.191	22.030	30.157	9.8

BUFFSIZE (字节)	real(s)	user(s)	sys(s)	吞吐(MB/s)
16	26.201	11.200	14.999	19.5
32	13.201	5.956	7.244	38.8
64	6.632	2.796	3.836	77.2
128	3.332	1.392	1.940	153.7
256	1.680	0.676	1.004	304.7
512	0.875	0.348	0.528	585.0
1024	0.460	0.192	0.268	1113.3
2048	0.269	0.087	0.182	1903.1
4096	0.169	0.044	0.124	3031.7
8192	0.120	0.028	0.092	4260.5
16384	0.109	0.012	0.096	4704.4
32768	0.089	0.008	0.080	5782.3
65536	0.078	0.000	0.078	6568.6
16777216	0.100	0.000	0.100	5138.7

4.2 stdio 逐字符读取（getc 宏 vs fgetc 函数）

方式	real(s)	user(s)	sys(s)	吞吐(MB/s)
getc()	1.769	1.604	0.165	289.4
fgetc()	1.589	1.448	0.141	322.2

4.3 fread() 库函数（不同 BUFFSIZE）

BUFFSIZE (字节)	real(s)	user(s)	sys(s)	吞吐(MB/s)
1	12.985	12.833	0.150	39.4
2	6.578	6.476	0.101	77.8
4	3.381	3.260	0.120	151.4
8	1.777	1.684	0.092	288.2
16	0.973	0.827	0.146	526.3
32	0.566	0.453	0.113	905.4
64	0.370	0.243	0.127	1384.2
128	0.274	0.169	0.105	1866.7

BUFSIZE (字节)	real(s)	user(s)	sys(s)	吞吐(MB/s)
256	0.226	0.099	0.127	2264.7
512	0.208	0.080	0.128	2456.3
1024	0.190	0.057	0.133	2696.5
2048	0.183	0.079	0.103	2801.4
4096	0.170	0.051	0.119	3008.6
8192	0.120	0.036	0.084	4266.0
16384	0.094	0.021	0.073	5425.4
32768	0.080	0.004	0.076	6376.4
65536	0.075	0.004	0.072	6799.1
16777216	0.096	0.000	0.096	5324.1

4.4 my_fread() 自实现 (内部缓冲固定 BUFSIZ, 不同请求大小)

请求大小 (字节)	real(s)	user(s)	sys(s)	吞吐(MB/s)
1	3.062	2.963	0.099	167.2
2	1.505	1.372	0.132	340.2
4	0.847	0.749	0.097	604.8
8	0.483	0.404	0.080	1059.1
16	0.301	0.200	0.100	1703.6
32	0.200	0.098	0.102	2554.2
64	0.165	0.048	0.116	3110.9
128	0.147	0.048	0.099	3485.2
256	0.135	0.041	0.094	3788.8
512	0.130	0.031	0.099	3942.3
1024	0.128	0.036	0.092	3998.8
2048	0.127	0.036	0.091	4035.1
4096	0.126	0.053	0.073	4070.3
8192	0.126	0.023	0.103	4053.3
16384	0.135	0.049	0.086	3793.9
32768	0.142	0.027	0.115	3607.0
65536	0.137	0.029	0.109	3724.1

请求大小 (字节)	real(s)	user(s)	sys(s)	吞吐(MB/s)
16777216	0.186	0.067	0.119	2754.8

4.5 write() 无 O_SYNC (不同 BUFFSIZE)

BUFFSIZE (字节)	real(s)	user(s)	sys(s)	吞吐(MB/s)
1	590.353	96.106	493.810	0.4
2	295.259	48.152	246.869	0.9
4	147.000	23.550	123.281	1.7
8	74.546	11.947	62.555	3.4
16	37.149	6.064	31.052	6.9
32	18.568	3.078	15.435	13.8
64	9.367	1.444	7.918	27.3
128	4.906	0.726	4.135	52.2
256	2.651	0.389	2.261	96.6
512	1.576	0.208	1.368	162.4
1024	0.863	0.096	0.764	296.7
2048	0.493	0.035	0.458	518.9
4096	0.298	0.028	0.270	859.7
8192	0.302	0.012	0.288	848.0
16384	0.251	0.009	0.243	1018.2
32768	0.290	0.004	0.286	881.4
65536	0.255	0.000	0.253	1002.5
16777216	0.216	0.000	0.216	1185.2

4.6 write() 有 O_SYNC (不同 BUFFSIZE)

注：受 `OSYNC_MAX_WRITES` 限制，各 BUFFSIZE 实际写入量不同，请同时记录 `bytes` 列。

BUFFSIZE (字节)	实际写入(bytes)	real(s)	user(s)	sys(s)	吞吐(MB/s)
1	4096	10.425	0.023	0.136	0.0004
2	8192	10.127	0.018	0.137	0.0008
4	16384	9.936	0.012	0.144	0.0016
8	32768	10.538	0.004	0.159	0.0030

BUFFSIZE (字节)	实际写入(bytes)	real(s)	user(s)	sys(s)	吞吐(MB/s)
16	65536	9.976	0.013	0.144	0.0063
32	131072	9.920	0.004	0.155	0.0126
64	262144	10.679	0.006	0.159	0.0234
128	524288	10.328	0.022	0.139	0.0484
256	1048576	10.270	0.018	0.146	0.0974
512	2097152	10.495	0.018	0.150	0.1906
1024	4194304	10.010	0.006	0.174	0.3996
2048	8388608	10.400	0.008	0.196	0.7693
4096	16777216	10.803	0.019	0.197	1.4811
8192	16777216	5.514	0.011	0.109	2.9016
16384	16777216	2.865	0.000	0.071	5.5855
32768	16777216	1.463	0.000	0.043	10.9367
65536	16777216	0.774	0.000	0.029	20.6800
16777216	16777216	0.031	0.000	0.016	522.8119

4.7 性能曲线图

下列图表由 `plot.py` 读取 `results.csv` 自动绘制（横轴均为 BUFFSIZE 的对数刻度）。重新生成方式见 3.6 节末尾的"绘图"说明。

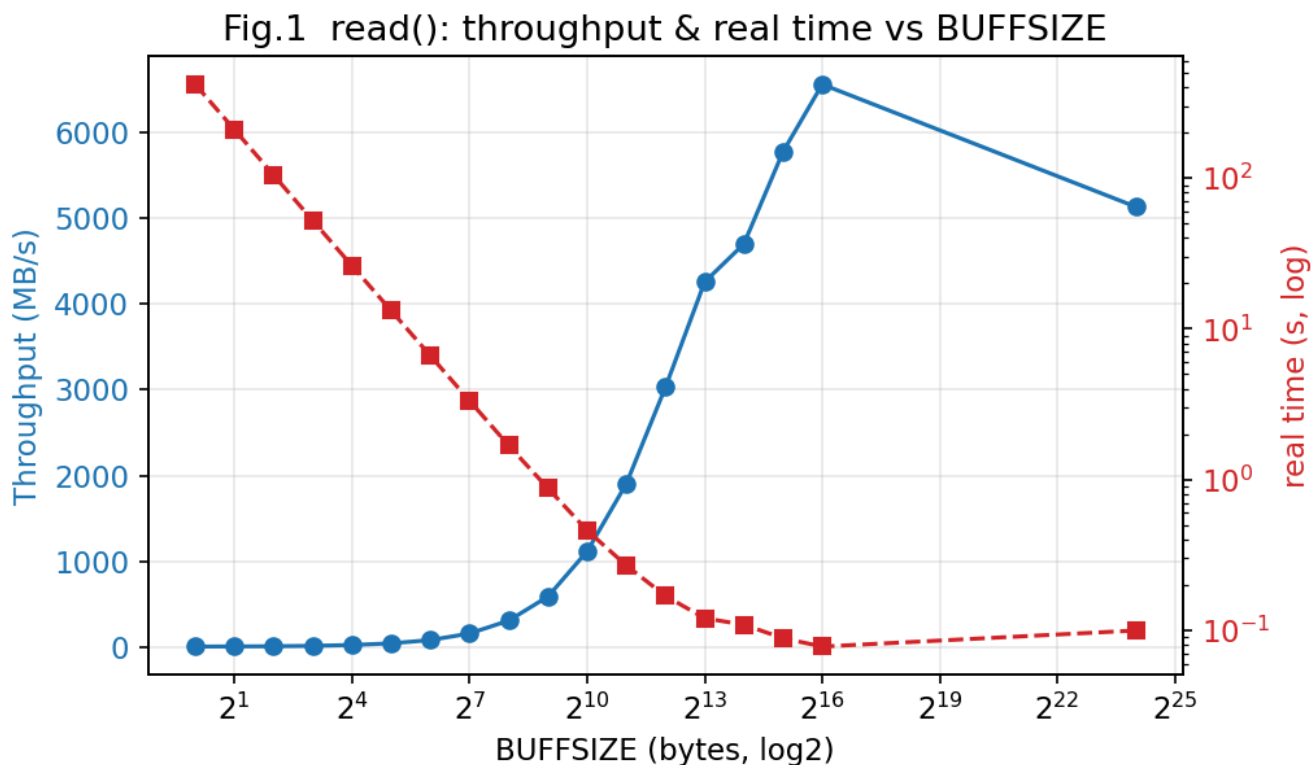


图1 `read()` 吞吐量（左轴，线性）与 real 时间（右轴，对数）随 `BUFSIZE` 的变化。吞吐量在 64 KB 处达峰约 6.6 GB/s，16 MB 反而回落；real 时间在小缓冲区段随 `BUFSIZE` 翻倍而减半，呈典型的"系统调用次数主导"反比关系。

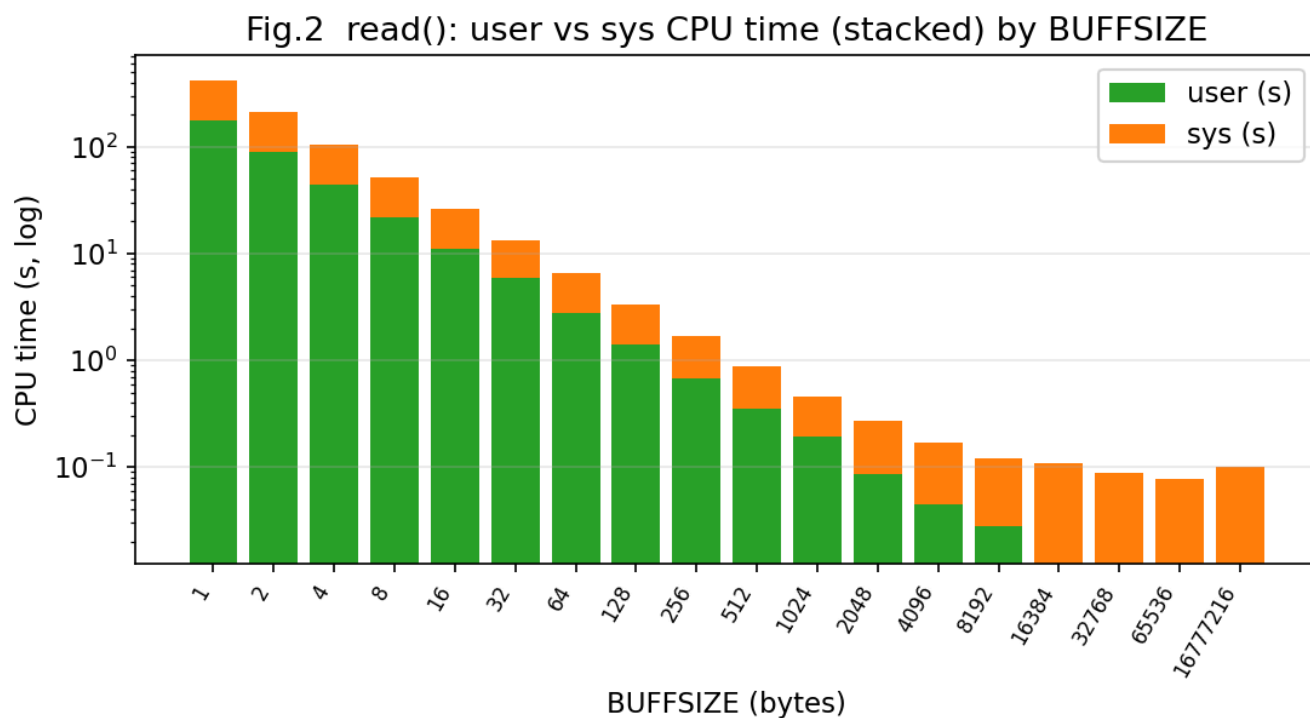


图2 `read()` 的 user 与 sys CPU 时间（堆叠，纵轴对数）随 `BUFSIZE` 的变化。小缓冲区段 user 与 sys 同步成倍下降；`BUFSIZE` ≥ 16384 后 user 时间趋近于 0，耗时几乎全部来自 sys（系统调用 + 内核态拷贝）。

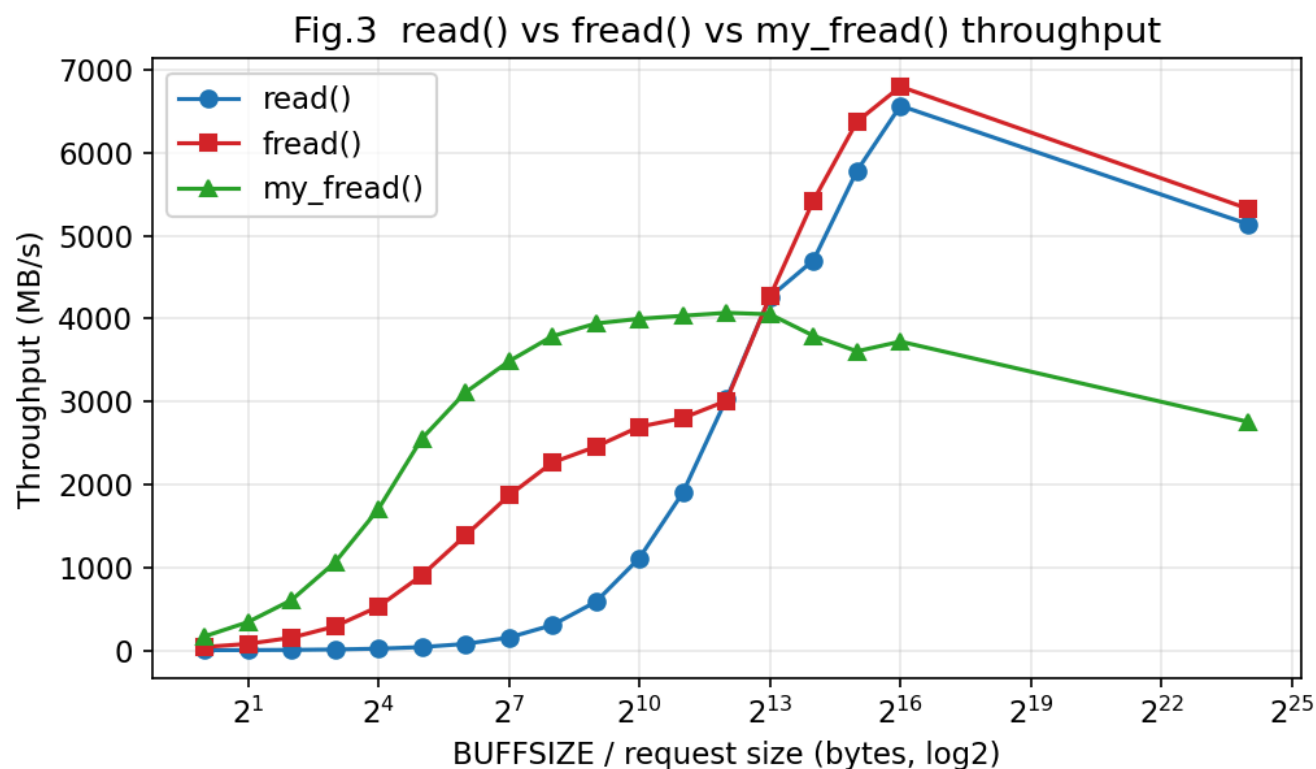


图3 `read()`、`fread()`、`my_fread()` 三者吞吐量对比。小粒度下 `my_fread/fread` 凭用户态缓冲远超裸 `read()`；中等粒度（约 8 KB）三者趋同；大粒度下 `my_fread` 受固定 8 KB 内部缓冲限制触顶约 4 GB/s，被能减少系统调用与拷贝的 `fread/read` 反超。

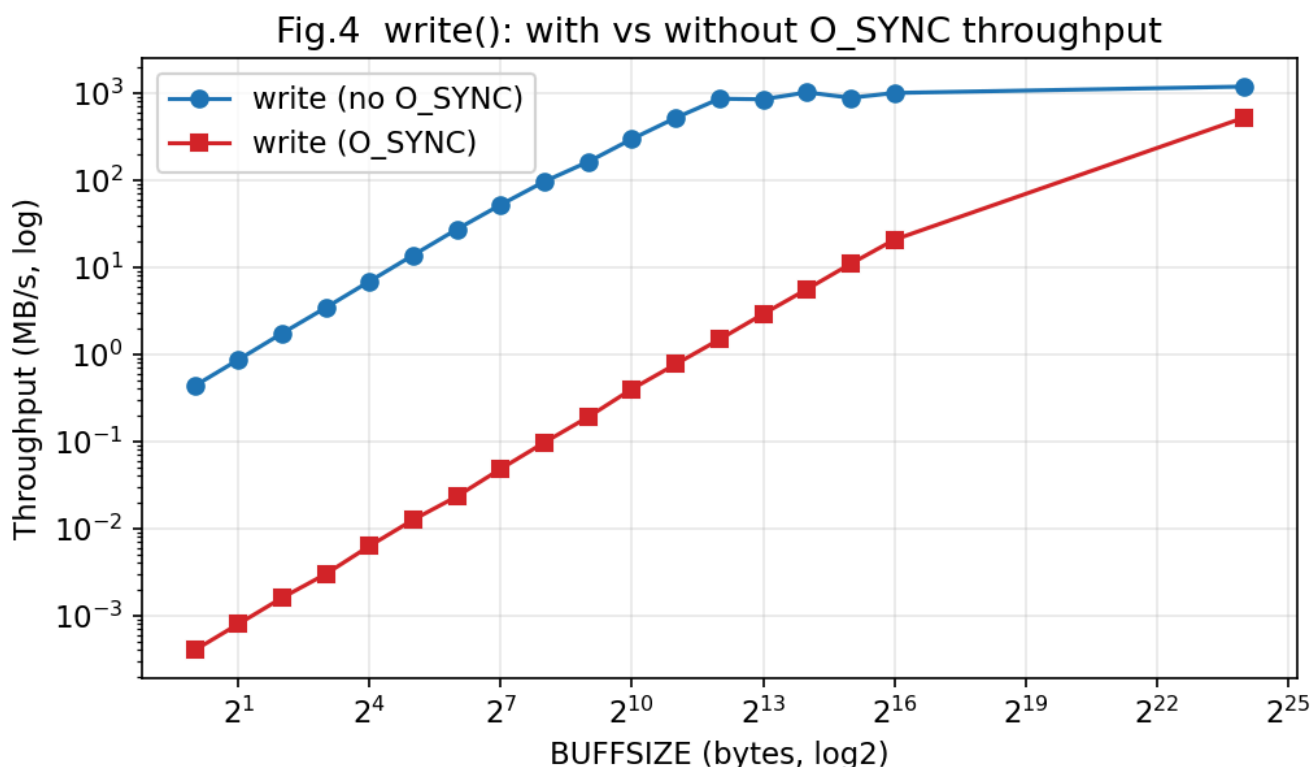


图4 `write()` 有无 `O_SYNC` 的吞吐量对比（纵轴对数）。无 `O_SYNC` 写入页缓存即返回，吞吐高出 `O_SYNC` 约两至三个数量级；`O_SYNC` 段吞吐随 `BUFSIZE` 近似线性上升，因耗时由 "同步落盘次数 × 单次约 2.4 ms 延迟" 决定（详见 5.4 节）。

5 数据分析

5.1 `read()` 随 `BUFSIZE` 的变化趋势

(1) 小缓冲区区间（1 ~ 512 字节）：完美的反比关系。观察 4.1 表，`BUFSIZE` 每翻倍，`real` 时间几乎精确减半：418.5 → 209.7 → 104.7 → 52.2 → 26.2 ...，吞吐量则成倍上升（1.2 → 2.4 → 4.9 → 9.8 MB/s）。原因是：读完 512 MB 文件所需的 `read()` 系统调用次数 = 512 MB / `BUFSIZE`，与 `BUFSIZE` 成反比；而此区间每次系统调用搬运的数据极少，总耗时几乎完全由系统调用次数决定。由 `BUFSIZE=1` 估算，单次 `read()` 系统调用约耗 418.5 s / 536,870,912 ≈ 780 ns（其中 `sys` 约 447 ns），这就是一次用户态/内核态往返的固定成本。

(2) 热缓存的有力佐证： $real \approx user + sys$ 。`BUFSIZE=1` 时 `user` 178.2 + `sys` 240.2 = 418.4 ≈ `real` 418.5；几乎没有“阻塞等待磁盘”的时间。这说明数据全部来自内核页缓存，CPU 一刻不停地在做“系统调用 + 拷贝”，耗时是计算/调用开销而非磁盘传输。也正因如此，峰值吞吐高达 6.5 GB/s——远超任何物理磁盘的顺序读速度，进一步证明读自页缓存。

(3) 拐点在文件系统块 / 页大小（4096）附近趋稳。当 `BUFSIZE` 达到 4096 字节（= 文件系统块大小 = 内存页大小）时吞吐已达 3032 MB/s，此后增益迅速放缓：4096 → 8192 → 16384 → 32768 → 65536 仅从 3032 升至 6569 MB/s。当单次调用搬运的数据足够多时，系统调用的“固定成本”已被摊薄到可忽略，瓶颈转移到内存拷贝带宽，曲线自然走平。

(4) 过大缓冲区 (16 MB) 反而变差。 65536 字节时吞吐达峰 6569 MB/s, 而 16 MB 时 回落到 5139 MB/s。原因是 16 MB 的用户缓冲区远超 CPU 的 L2/L3 缓存容量, 每次 `read()` 拷贝目标地址都落在冷缓存上, 访存效率下降; 同时大缓冲带来的 TLB/缓存抖动抵消了"减少 系统调用次数"的收益。这说明缓冲区并非越大越好, 最优值在几十 KB (能装进 CPU 缓存 且已充分摊薄系统调用) 量级。

5.2 `read()` vs `getc()/fgetc()` vs `fread()` 的对比

把"每次只取 1 字节"的几种方式放在一起 (见下表), 差异极其悬殊:

方式 (1 字节粒度)	real(s)	sys(s)	吞吐(MB/s)	相对 read=1
read (BUFSIZE=1)	418.512	240.240	1.2	1×
fread (size=1)	12.985	0.150	39.4	≈ 33×
my_fread (req=1)	3.062	0.099	167.2	≈ 137×
getc()	1.769	0.165	289.4	≈ 237×
fgetc()	1.589	0.141	322.2	≈ 264×

关键在 **sys 时间**。逐字节 `read()` 触发 5.4 亿次系统调用, sys 高达 240 s; 而其余四者 都带用户态缓冲, 对 512 MB 文件只触发约 **512MB / 缓冲区 次 `read()`** (stdio 默认缓冲 ≈ 8 KB → 约 6.5 万次), sys 骤降到 0.1~0.17 s (相差三个数量级)。这就是用户态缓冲 的全部威力: 把昂贵的内核往返摊薄上万倍。

同是"带缓冲取单字节", 为何 `getc` 又快于 `fread`? 注意 `getc/fgetc/fread` 的 sys 时间 都已很小且相近 (系统调用次数一致), 差距全在 **user 时间**: `fread@1` 的 user 高达 12.8 s, 而 `getc` 仅 1.6 s。这是因为 `fread(buf, 1, 1, fp)` 每取 1 字节都要完整走一遍 `fread` 的逻辑 (计算 `size×nmemb`、循环、`memcpy` 准备等), 单字节调用下这些"每次调用的额外指令"被 放大了; 而 `getc()` 是为"取一个字符"高度优化的精简路径, 几乎只是"缓冲区取一字节 + 移动 指针"。结论: 减少系统调用解决了 **sys 开销**, 但每次库调用自身的 **user 态开销**在极小粒度下 依然重要, 专用接口 (`getc`) 优于通用接口 (`fread`) 的退化用法。

`fgetc` 与 `getc` 几乎相同。见 5.3 之外的对比: `fgetc`(322 MB/s) 甚至略快于 `getc` (289 MB/s), 差异在测量抖动范围内, 详见 Q2。

5.3 `fread()` vs `my_fread()` 的对比

(1) 极小粒度下 `my_fread` 反而更快。 req=1 时 `my_fread` 167 MB/s, 远高于 `fread` 的 39 MB/s (见 5.2 表)。原因同上: 自实现的 `my_fread` 路径简单 (缓冲区取字节 + 计数), 而 glibc `fread` 为通用性付出了更多每次调用开销。这说明"标准库总是最快"是误解——在 退化用法下, 精简的自实现可能胜出。

(2) 中等粒度 (数百字节 ~ 8 KB) 两者接近。 在 8192 字节处 `fread` 4266 MB/s、`my_fread` 4053 MB/s, 相差不到 5%, 都接近 `read` 的 4260 MB/s。此时系统调用已被充分摊薄, 三者拼的是内存拷贝, 自然趋同。这验证了缓冲机制的本质: 把系统调用次数降到 **文件大小 / 内部缓冲区大小**, 与请求粒度无关。

(3) 大粒度下 `my_fread` 触及天花板 (约 4000 MB/s), 不及 `fread/read`。 请求 64 KB 时 `fread` 6799、`read` 6569, 而 `my_fread` 仅 3724 MB/s; 16 MB 时差距更大。根本原因是 `my_fread` 的内部缓冲区固定为 **BUFSIZ(8192)**: 无论调用者请求多大, 它每次最多向内核 `read()` 8192 字节, 因此系统调用次数恒为 **512MB/8192 ≈ 65536** 次, 无法随请求增大而下降; 而且数据要经历"页缓存 → my 缓冲区 → 用户缓冲区"两次拷贝。glibc `fread` 与裸 `read()` 在大请求时能够 (接近) 直接把大块数据搬入用户缓冲、显著减少系统调用与拷贝, 于是反超。这恰好说明: 缓冲机制的效率上限由内部缓冲区大小决定, 而非外部请求大小。

5.4 write() 有无 O_SYNC 的对比

(1) 无 O_SYNC: 与 read 同构的"系统调用次数主导"曲线。见 4.5 表, BUFFSIZE 翻倍 real 减半 (590 → 295 → 147 ...), sys 时间占绝对主导 (BUFFSIZE=1 时 sys 493.8 s, user 仅 96.1 s), 约 4096 字节后走平, 峰值约 1185 MB/s。write() 在无 O_SYNC 时 只是把数据拷入内核页缓存便立即返回 (延迟写 / write-back), 由内核稍后异步回写磁盘, 因此极快。值得注意的是写峰值 (约 1.2 GB/s) 明显低于读峰值 (约 6.5 GB/s), 因为写路径 还要分配页帧、标记脏页、维护回写结构, 单字节 write() 的 sys 成本 (约 1.84 μs) 也高于 read() (约 0.45 μs)。

(2) 有 O_SYNC: 慢三个数量级, 由磁盘同步延迟决定。同为 BUFFSIZE=4096, 无 O_SYNC 859.7 MB/s vs 有 O_SYNC 1.48 MB/s, 相差约 580 倍; BUFFSIZE=1 时更是 0.43 vs 0.0004 MB/s (约 1000 倍)。O_SYNC 要求每次 write() 都等数据真正落到存储设备才返回。

(3) "10 秒平台"揭示同步延迟。 在 BUFFSIZE 1~4096 区间, real 时间几乎恒为约 10 s 而与 BUFFSIZE 无关——因为 O_SYNC_MAX_WRITES=4096 把写调用次数锁定在 4096 次, 每次都强制同步落盘。由此可估出单次同步落盘延迟 $\approx 10\text{ s} / 4096 \approx 2.4\text{ ms}$ 。这就是为什么此 区间吞吐量随 BUFFSIZE 近似线性增长: 耗时由"同步次数 × 单次延迟"决定, 次数固定, 每次多 写一些 (BUFFSIZE 更大) 数据, 单位时间搬运量自然成比例上升。

(4) BUFFSIZE ≥ 8192: 写满 16 MB 上限所需的同步次数开始下降。此时 $16\text{MB} / \text{BUFFSIZE} < 4096$, 同步次数随 BUFFSIZE 反比下降, real 随之跌落 (8192→5.5 s, 65536→0.77 s); 极端地, 16 MB 一次写完只需 1 次同步, 0.031 s、522 MB/s。这再次印证: O_SYNC 下性能瓶颈是同步落盘的次数与设备延迟, 减少同步次数 (增大每次写 入量) 是唯一有效的优化手段。注意此处约 2.4 ms 的同步延迟介于典型 HDD (~10 ms) 与本地 SSD (亚毫秒) 之间, 符合 2 节判断——底层为虚拟化/网络块存储, 而非真正的旋转磁盘。

6 思考与讨论

Q1: stdio 内部维护了用户态缓冲区, 为什么逐字符调用 getc() 的性能仍远优于 逐字节调用 read()?

实测 getc 289 MB/s vs read@1 1.2 MB/s, 快约 237 倍。根因是系统调用次数: 逐字节 read() 触发 5.4 亿次内核往返, 每次都要保存/恢复上下文、切换特权级、可能冲刷 TLB 与 缓存, sys 时间高达 240 s; 而 getc() 绝大多数调用只是从 stdio 的约 8 KB 用户态缓冲区 取一个字节、移动指针, 仅在缓冲区耗尽时才触发一次 read(), 对 512 MB 文件约只有 6.5 万次系统调用, sys 降到 0.165 s。缓冲把昂贵的内核往返摊薄了约 1/8192, 这正是性能差距的 来源。

Q2: getc() 宏实现与 fgetc() 函数实现之间是否存在性能差异?

理论上 getc() 常以宏实现可省一次函数调用, 应略快于函数 fgetc()。但本次实测 fgetc 322 MB/s 反而略快于 getc 289 MB/s (user 1.448 vs 1.604 s), 差异约 10%, 处于 单次测量的抖动范围内, 可视为无显著差异。原因有二: 其一, 现代 glibc 中 getc 未必 是真正的宏, 且二者最终都走加锁的 _IO_getc 路径; 其二, -O2 下编译器对函数调用做了 充分优化, 一次额外调用的开销相对"每字节取数 + 缓冲管理"占比极小。结论: 在现代 glibc + 优化编译下, 宏/函数之分对单字符读取已基本不可观测。

Q3: fread() 与 read()、getc() 相比, stdio 缓冲机制起到了什么作用?

fread 同时受益于两点: ①调用者可一次请求大块 (批量请求), ②FILE 对象内部仍有缓冲。小粒度下 (见 5.2 表), fread@1 (39 MB/s) 相对 read@1 (1.2 MB/s) 快约 33 倍, 靠的就是 内部缓冲把系统调用从 5.4 亿次降到约 6.5 万次 (sys 0.15 s vs 240 s)。但 fread@1 又慢于 getc(289 MB/s), 因为 fread(..., 1, 1, ...) 通用路径每字节的 user 态开销远高于专用的 getc。当请求增大到 ≥8 KB, fread 追平并略超 read (6799 vs

6569 MB/s)。作用总结：缓冲机制把"系统调用次数"这个主导因素压到最低；在此之上，接口是否针对粒度优化决定剩余的 **user** 态差距。

Q4: **my_fread()** 与 **fread()** 的对比说明了什么？

自实现验证了缓冲的本质——系统调用次数 = 文件大小 / 内部缓冲区大小，与调用者请求粒度 无关：故 **my_fread** 在 req=1 时 (167 MB/s) 就远胜 **read@1**，并在中等粒度 (8 KB) 追平 **fread** (4053 vs 4266 MB/s)。但 **my_fread** 的内部缓冲固定为 **BUFSIZ**(8192)，系统调用次数恒为 约 6.5 万次且数据要多一次拷贝，因此无法随请求增大继续提速，峰值卡在约 4000 MB/s，被能减少系统调用/拷贝的 **fread**、**read** (约 6700 MB/s) 反超。说明：简单缓冲即可逼近库的中段性能，但库在大块路径上的优化 (更大/可变缓冲、减少拷贝) 才是其峰值优势所在。

Q5: 有无 **O_SYNC** 的性能差异说明了内核缓冲区 (页缓存) 的什么作用？

同 **BUFSIZE=4096** 下无 **O_SYNC** 859.7 MB/s、有 **O_SYNC** 1.48 MB/s，相差约 580 倍。无 **O_SYNC** 时 **write()** 只把数据拷入内核页缓存即返回，由内核异步批量回写，故 $real \approx user + sys$ (纯 CPU 拷贝，无磁盘等待)；**O_SYNC** 则每次写都必须等数据落到设备，**real** (约 10 s) 远大于 **user+sys** (约 0.16 s)，绝大部分时间是阻塞等待磁盘同步 (单次约 2.4 ms)。这凸显页缓存的核心价值：用"延迟写 + 合并回写"换取数量级的吞吐提升。代价是可靠性：掉电/崩溃时页缓存中尚未回写的数据会丢失；**O_SYNC** (或 **fsync**) 以性能为代价 换取"返回即落盘"的持久性保证，二者是经典的性能/可靠性权衡。

Q6: 综合讨论——系统调用开销与缓冲的意义。

贯穿全部数据的主线是一条：**I/O 性能的首要瓶颈是系统调用次数，而非单次传输量**。读、写、**O_SYNC** 三组曲线在小缓冲区段都呈"**BUFSIZE** 翻倍、耗时减半"的反比规律，正因为 $耗时 \propto 系统调用次数 \propto 1/BUFSIZE$ 。缓冲 (无论是 **stdio**、**my_fread** 还是内核页缓存) 的本质都是把多次小 I/O 合并成少数大 I/O，摊薄固定的内核往返/同步成本。但缓冲并非越大越好：**read** 在 64 KB 达峰后，16 MB 反而回落 (5.1(4))，因为超出 CPU 缓存 导致访存效率下降。最优缓冲区是"足够大以摊薄系统调用、又足够小以驻留 CPU 缓存"的折中，本实验落在几十 KB 量级 (与文件系统块 4096 字节为同一数量级的若干倍)。因此 I/O 优化的 两条核心原则是：①尽量减少系统调用次数 (用缓冲/批量)；②缓冲区取数十 KB 量级，而非盲目求大。

7 结论

本实验在云虚拟机 (Ubuntu 22.04 / 内核 5.15、ext4、4 核、页与块大小均为 4096 字节) 上、热缓存条件下，系统对比了多种 UNIX/Linux I/O 方式，主要结论如下：

1. 系统调用次数是 I/O 性能的首要决定因素。在小缓冲区段，读、写、**O_SYNC** 三类测试 均呈"**BUFSIZE** 翻倍、耗时减半"的反比规律；逐字节 **read()** 因 5.4 亿次系统调用而最慢 (1.2 MB/s)，其 **sys** 时间高达 240 s。
2. 用户态缓冲能把系统调用开销摊薄三个数量级。同为单字节读取，带缓冲的 **getc/fgetc/fread/my_fread** 比裸 **read()** 快 33~264 倍，**sys** 时间从 240 s 降到 0.1 s 量级。
3. 接口与缓冲区大小共同决定上限。极小粒度下专用接口 **getc** (289 MB/s) 优于通用 **fread@1** (39 MB/s)；自实现 **my_fread** 受固定 8 KB 内部缓冲限制，峰值约 4000 MB/s，被可减少拷贝/系统调用的 **fread**、**read** (约 6700 MB/s) 反超。
4. 性能排序 (峰值吞吐，热缓存读)：**fread**(6799) $\approx$ **read**(6569) $>$ **my_fread**(4070) $\gg$ **fgetc/getc**(约 300)；写入：无 **O_SYNC**(1185 MB/s) $\gg$ 有 **O_SYNC**(同 **BUFSIZE** 下相差约 580~1000 倍)。

5. 最佳 **BUFFSIZE** 在数十 KB 量级。读吞吐在 4096 (=块/页大小) 后迅速趋稳、在 64 KB 达峰, 16 MB 因超出 CPU 缓存反而回落。缓冲区"足够大以摊薄系统调用、又足够小以驻留 CPU 缓存"是最优折中。
6. 内核页缓存以可靠性换吞吐。无 **O_SYNC** 的延迟写让 **write()** 拷入页缓存即返回 ($real \approx user + sys$, 无磁盘等待); **O_SYNC** 每次强制落盘 (单次约 2.4 ms), $real$ 由设备 同步延迟主导。这是性能与持久性的经典权衡。

适用性与局限: (1) 数据为热缓存、单次运行, 读测试反映的是"页缓存→用户缓冲"的拷贝与系统调用开销, 不代表物理磁盘冷读性能; 如需评估磁盘本身, 应加 **-d** 或 **drop_caches** 在冷缓存下重测, 并多次取平均。(2) 运行于虚拟机, I/O 路径经虚拟化层, **rotational=1** 不反映真实介质 (实测同步延迟约 2.4 ms 表明底层为虚拟化/网络块存储而非 HDD); 绝对数值在不同宿主/云盘上会有差异, 但上述定性规律与相对关系具有普遍性。(3) 程序每项只测一次, 存在偶发抖动 (如 **getc/fgetc** 的细微反常), 结论以数量级与趋势为准。

参考文献

1. W. Richard Stevens, Stephen A. Rago. *Advanced Programming in the UNIX Environment* (APUE). 第 3 章 (文件 I/O)、第 5 章 (标准 I/O 库)、第 14 章 (高级 I/O) .